PSG College of Technology, Coimbatore
Department of Computer Science and Engineering

Automated Low-Resource On-Device Classroom Attendance via Sweep-Video Face Detection

Project Work (23Z711) | Review III

Guided by Ms Sneha C Co-guided by Dr Arul Anand

Ananya R S (23Z306)
Athish Pranav (23Z310)
Dileepan U S (23Z319)
Sanjeev S (23Z361)
Sreeharini Ganishkaa S (23Z370)

Introduction

Problem

Manual roll-call takes up lecture time and allows proxy attendance. Camera-based systems in the literature assume fixed cameras, clear photos or a GPU server.

Our approach

The teacher records a 10 to 15 second sweep of the classroom on a phone. The app detects, tracks and recognises every student on the phone itself, offline and on the CPU, and produces the attendance list.

Progress

- iOS app complete with a native C++ engine
- Android app complete with an ML Kit + ONNX engine
- Multi-pose enrollment, CSV export, roster sharing
- Benchmark mode with a live detector switch

Market Survey

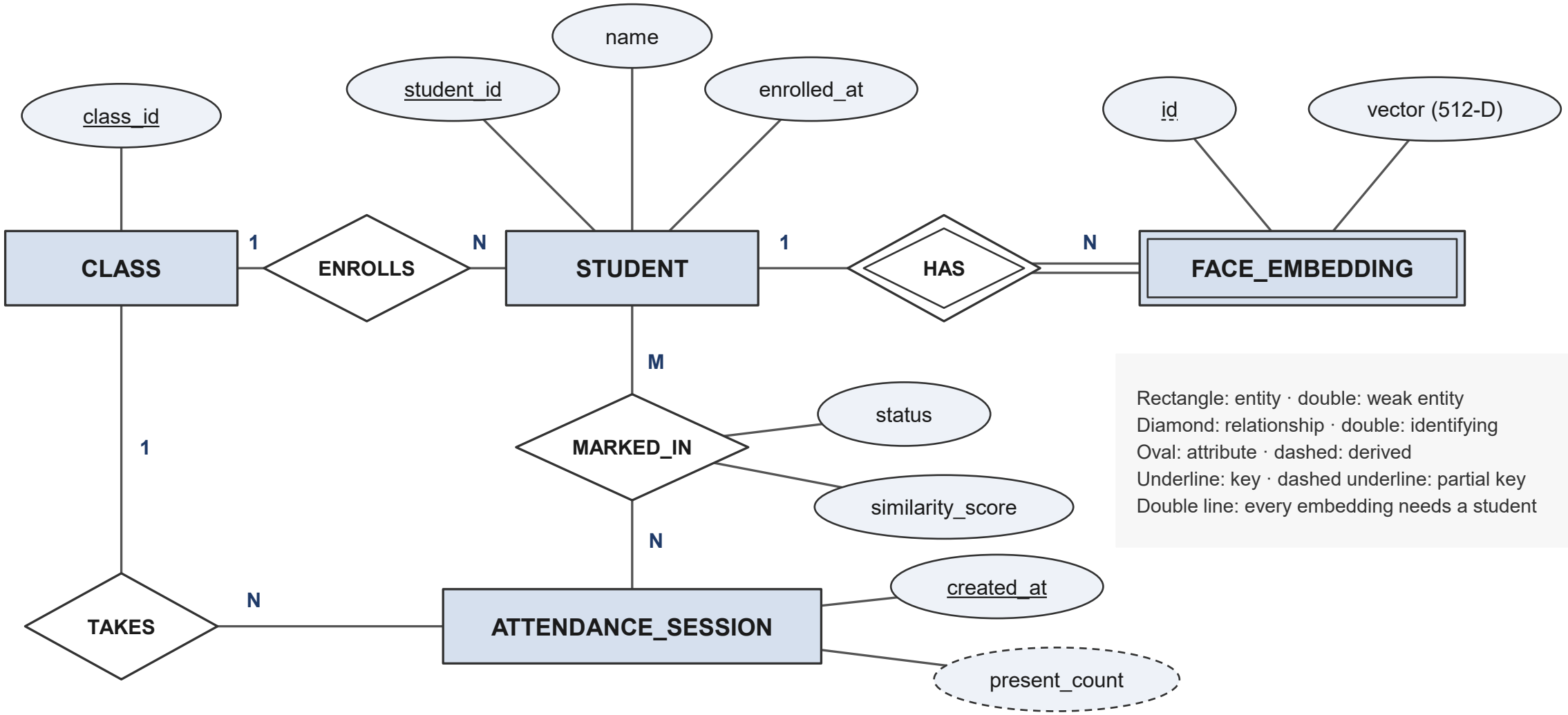
Approach	Example in use	Whole class at once	Extra hardware	Main drawback
Manual roll-call / register	Most classrooms	No	None	Slow, and proxies are easy. Teachers in Tamil Nadu schools spent over 45 min a day on it
Fingerprint terminal	Common in colleges and offices	No	Terminal per room or gate	Students queue one at a time; shared touch surface
Face-recognition device at entry	TNeGA DL-FRS, Tamil Nadu public schools	No	Edge device with camera	Marks arrival at the gate, not presence in a lecture
Face-recognition mobile app	AttendFy (App Store)	No	None	Captures faces one by one
CCTV + cloud processing	Real-time tracking + Firebase	Yes	Fixed camera per room + server	Costly per room; student video leaves campus
Our system	ClassRoom app	**Yes**	**None (teacher's phone)**	Accuracy varies based on classroom environmental factors

Gap: no common solution captures a whole class at once without extra hardware and without sending data off the device.

Progress

Planned	Status	What was done
Multi-device test setup	**Done**	Running on iPhone (iOS 26.6) and OnePlus CPH2661 (Android 16)
Off-angle and side-profile faces	**Done**	3-pose enrollment (frontal, left 15°, right 15°)
Benchmark more detectors	**Done**	RetinaFace and Haar added with a live switch; speed measured on CPU
Rear rows and occlusion	**Done**	Rear-row zoom and desk tracking added to both pipelines
Classroom vs lab tests, look-alikes	**Done**	Tested in classroom and lab settings, including look-alike cases
Deployment packaging	**Done**	Release builds prepared for iOS and Android
Multi-sweep capture	**Done**	Several sweeps can be recorded for the same lecture
Consolidated attendance	**Done**	Results from all sweeps merged into one attendance list

ER Diagram



Rectangle: entity · double: weak entity
Diamond: relationship · double: identifying
Oval: attribute · dashed: derived
Underline: key · dashed underline: partial key
Double line: every embedding needs a student

Assumptions

Teacher and device

- The teacher already owns a smartphone (iOS 16+ or Android 7.0+)
- It has a working rear camera that records 1080p video and a gyroscope
- About 200 MB of free storage; no internet connection is needed

Students and enrollment

- Every student is enrolled once, in person and with consent (3 face poses)
- Roll numbers are unique within a class
- Faces are mostly uncovered and roughly facing the front during the sweep

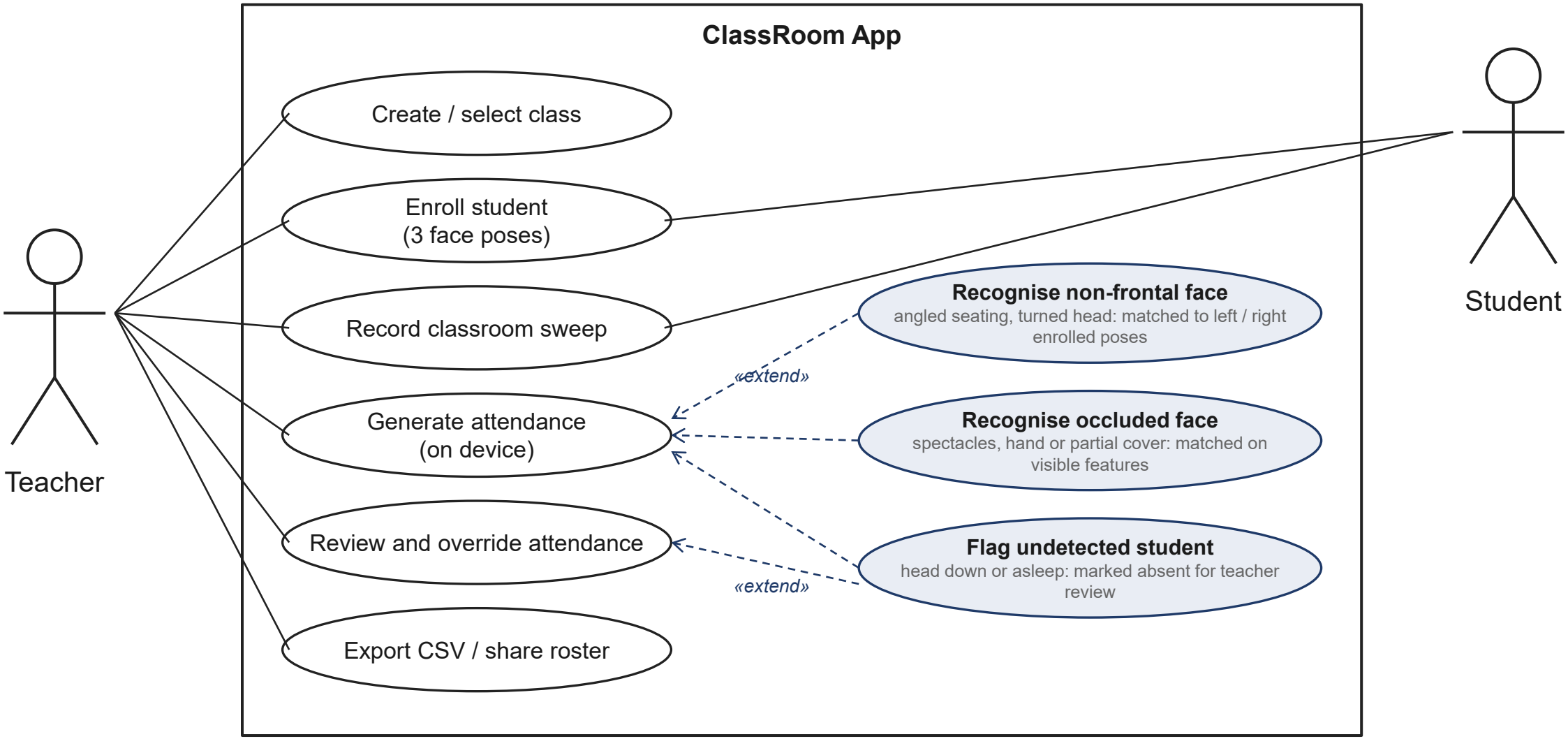
Classroom

- Students sit in rows facing the teacher, who stands at the front
- Normal indoor lighting, not a dark room
- Back rows appear in the upper part of the frame (the rear-row zoom relies on this)

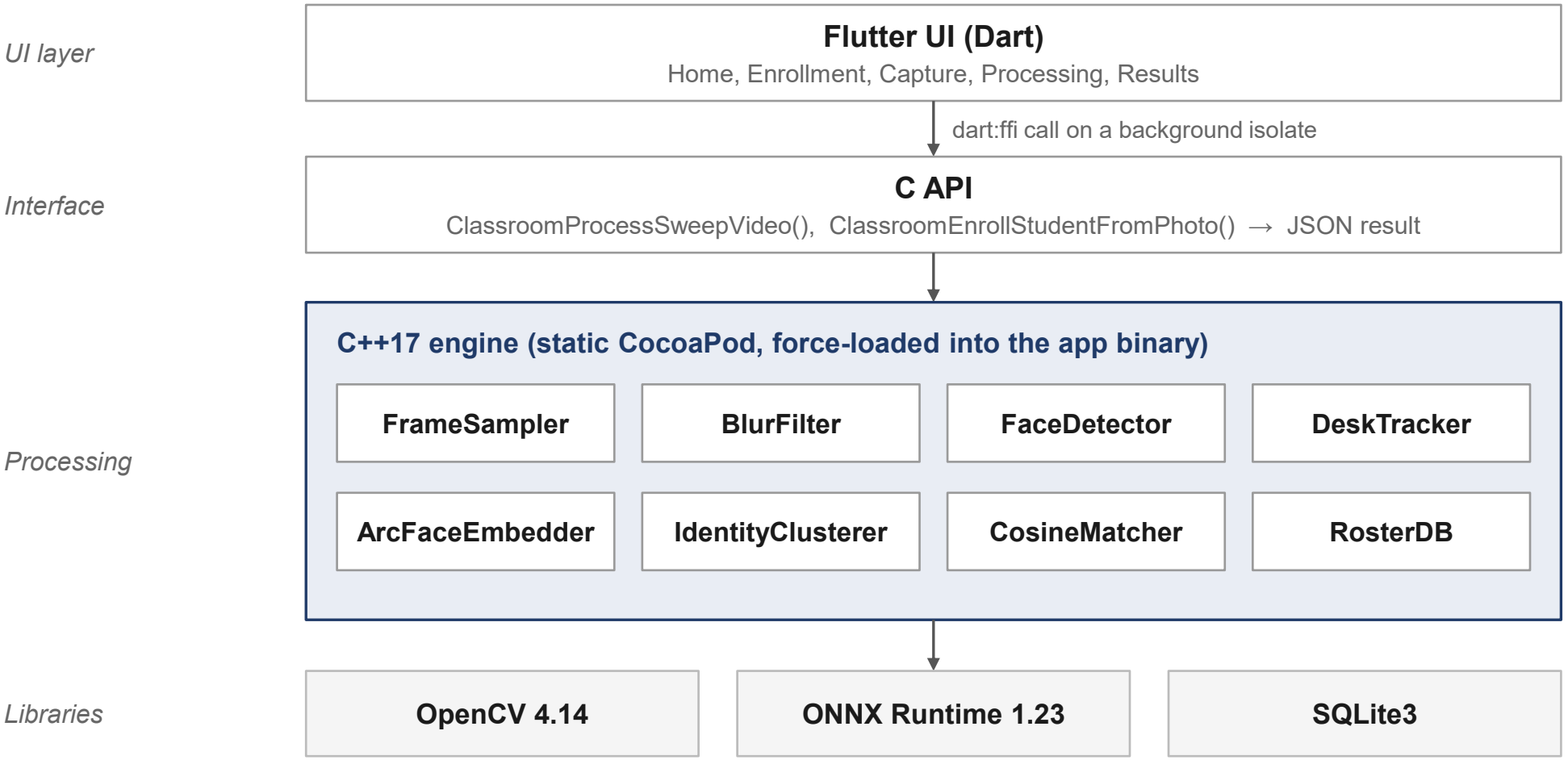
How it is used

- The teacher pans slowly for 10 to 15 s, following the on-screen speed warning
- One sweep per lecture, taken while students are seated
- The teacher reviews the list and corrects errors before exporting

Use Case Diagram

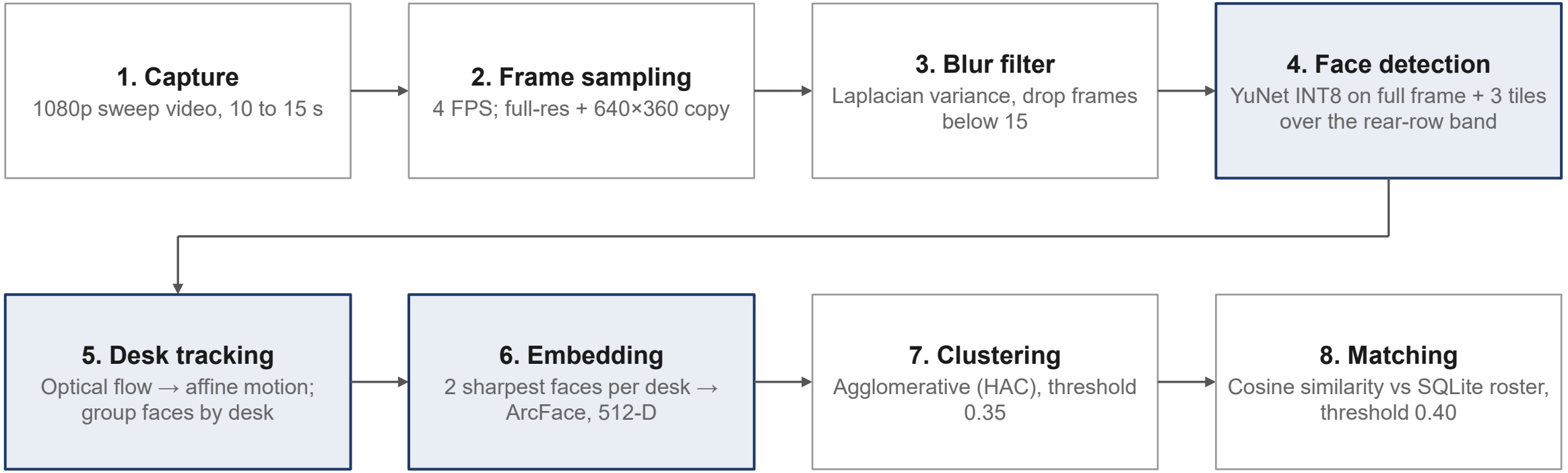


iOS Implementation: Architecture



Runs on the iPhone CPU only (no GPU / Neural Engine)

iOS Implementation: Pipeline

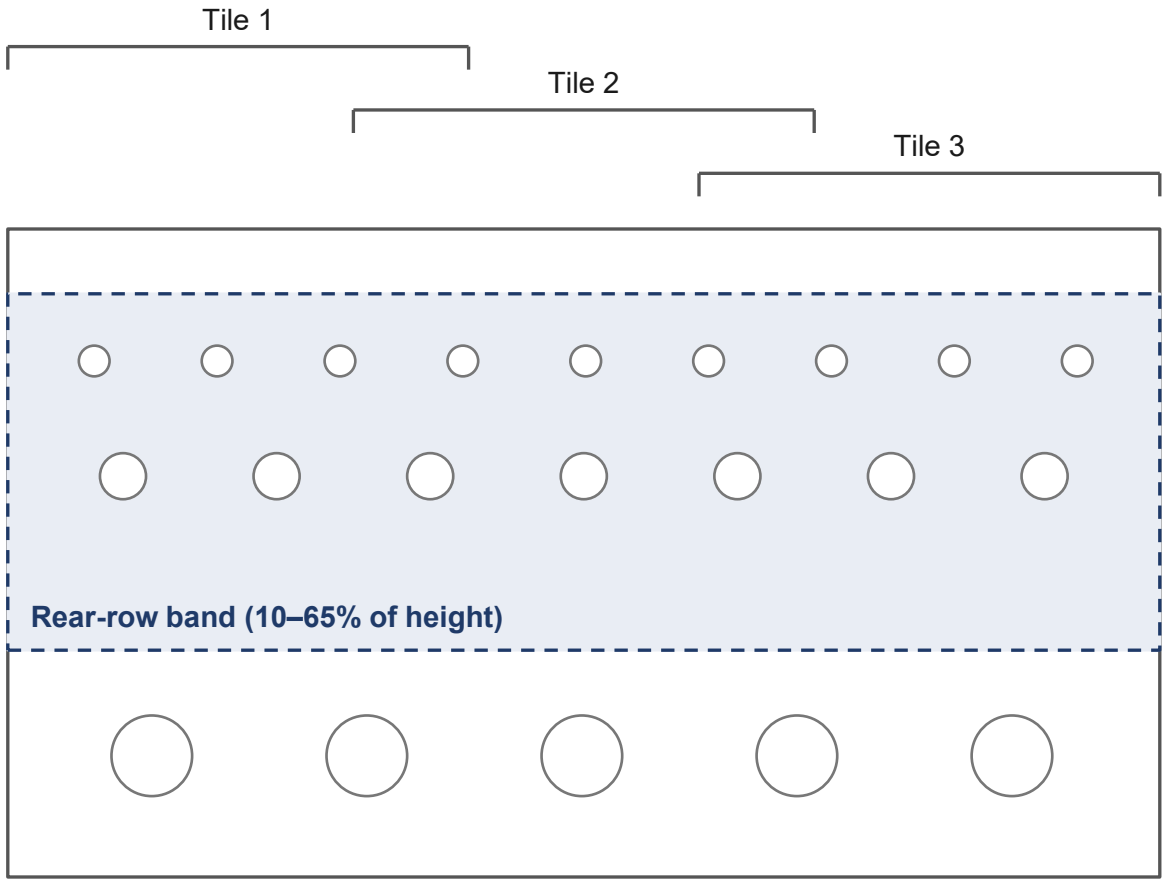


Rear-row zoom: extra passes only on the band where the back rows sit, not the whole frame.

Desk tracking: the moving camera sees past occluding heads; sightings of one student are grouped by desk.

Best-2 embedding: only the 2 sharpest faces per desk go to ArcFace, instead of every face in every frame.

Face Detection Engine



Front rows: found in the full-frame pass

Measured on CPU: 6.1 ms per frame (full frame only), 30.3 ms with the rear-row tiles.

YuNet INT8

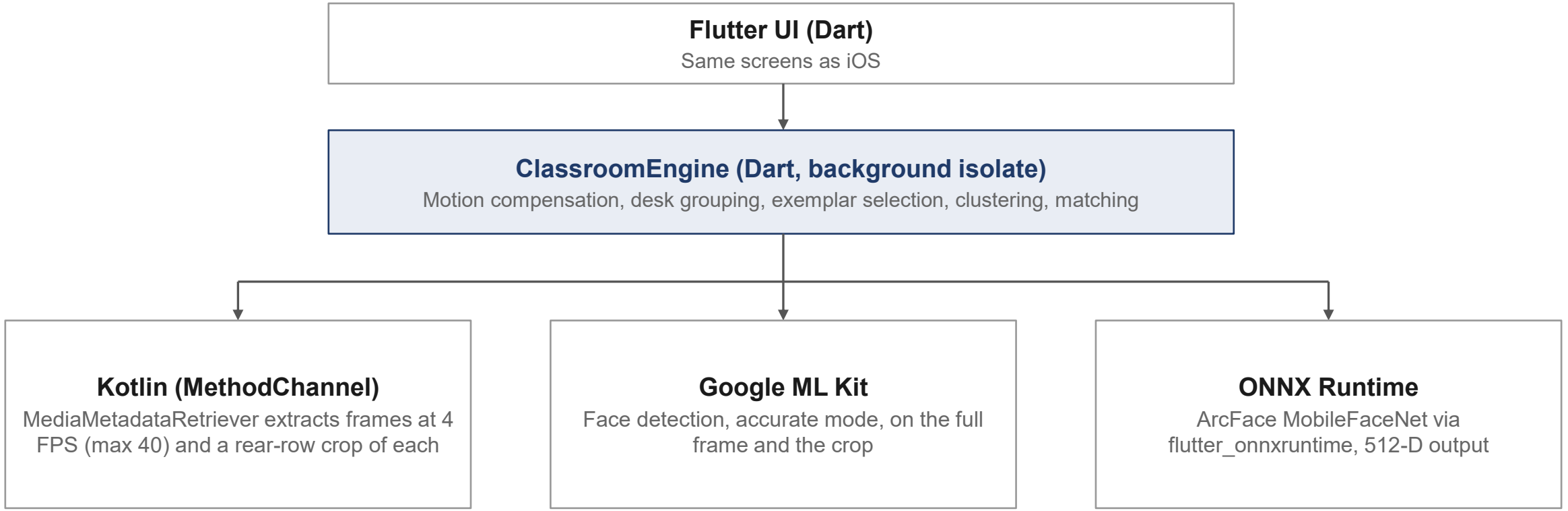
Light CNN face detector, 0.1 MB. Gives a box and 5 landmarks per face.

1. **Full-frame pass** on a 640×360 copy, score ≥ 0.40
2. **Rear-row pass** 3 tiles over the back-row band, each enlarged to 640 px, score ≥ 0.35
3. **Merge** map boxes to 1080p, drop overlaps (IoU ≥ 0.35) and faces under 14 px
4. **Hand-off** landmarks used to align each face for ArcFace

Why only a band? Back-row faces shrink to a few pixels. Tiling the whole frame (SAHI) costs too much on a phone, so we zoom only where the back rows sit.

Also: auto-rotation for sideways video; RetinaFace or Haar selectable at runtime. Android uses Google ML Kit with the same rear-row crop idea.

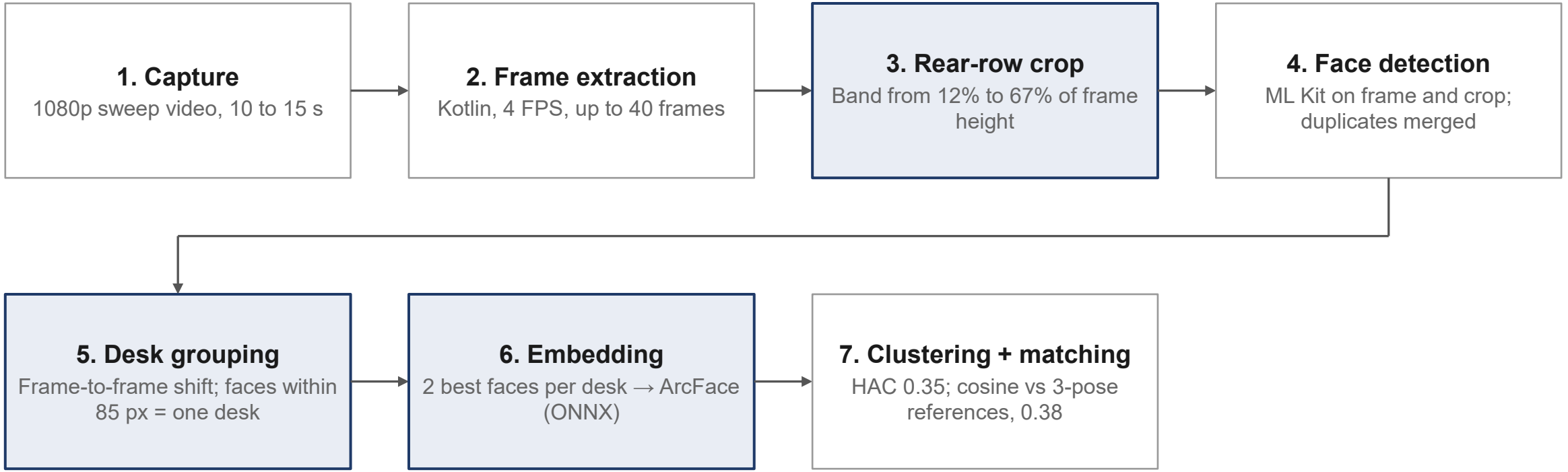
Android Implementation: Architecture



Storage: JSON roster per class in app storage, CSV export

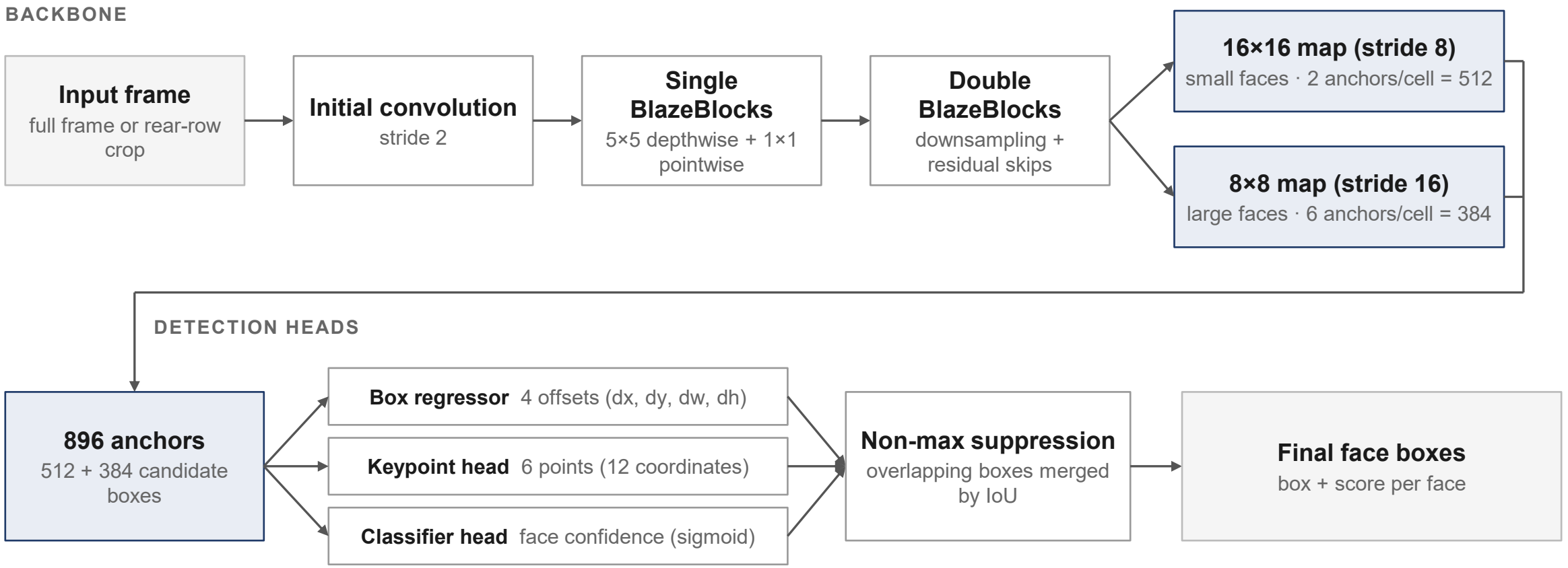
Tested on: OnePlus CPH2661, Android 16, CPU only

Android Implementation: Pipeline



	iOS	Android
Frame decoding	OpenCV, in C++	MediaMetadataRetriever, in Kotlin
Face detector	YuNet INT8 (RetinaFace / Haar selectable)	Google ML Kit
Camera motion	Optical flow + affine fit	Global translation estimate
Roster storage	SQLite	JSON file

Face Detection on Android: ML Kit (BlazeFace)



In our app: ML Kit runs twice per frame, on the full frame and on the 12–67% rear-row crop (accurate mode, min face 1% of width). Boxes are merged (IoU ≥ 0.35, ≥ 14 px) and sent to ArcFace.

Architecture from Bazarevsky et al., "BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs", 2019. Google does not publish ML Kit's exact internal model.

How Data Is Stored

On the teacher's phone (app's private storage)

models/	YuNet, RetinaFace, Haar, ArcFace (16.4 MB), copied from the app on first launch
classes.json	List of the teacher's classes
roster_<class>.db (iOS)	SQLite: students table + embeddings table
roster_<class>.json (Android)	Roll no., name, enrol time and 3 embeddings per student

Temporary (cleared or replaced)

sweep_frames/ (Android)	Extracted frames; emptied at the start of every run
attendance_<time>.csv	Created only when the teacher taps Export

Stored per student

- Roll number and name
- 3 face embeddings, 512 numbers each

Not stored

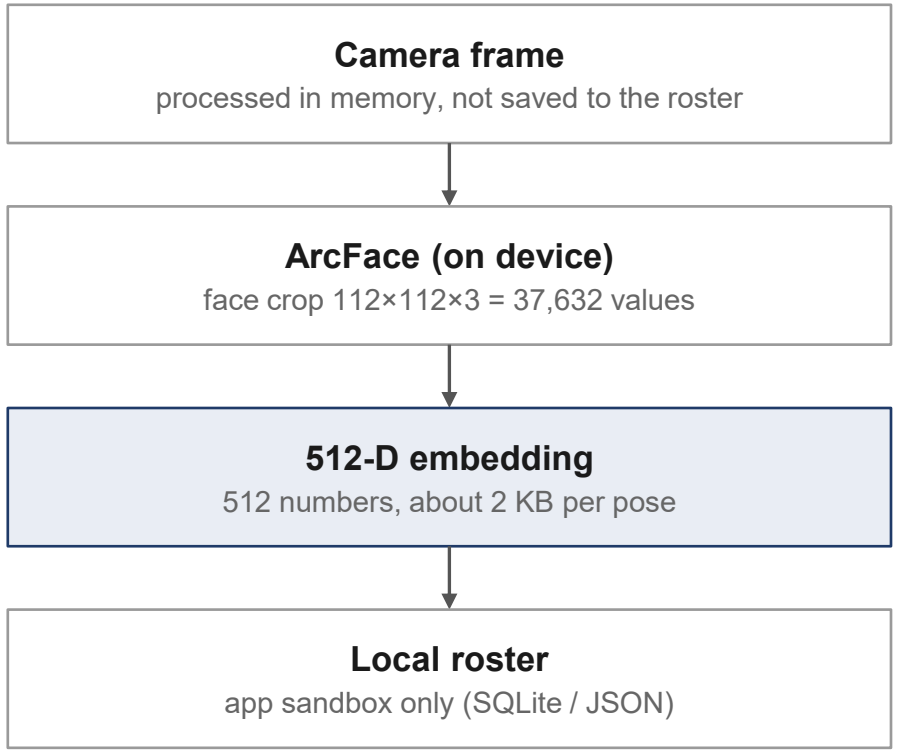
- Sweep frames: processed in memory on iOS, cache cleared each run on Android
- Nothing is uploaded to any server

Leaves the phone only when the teacher shares

- Attendance CSV: student ID, name, status, score
- Roster JSON: to hand a class over to another teacher

Privacy and Biometric Data Protection

What is kept



Can a face be rebuilt from a stored embedding?

One-way by design. 37,632 pixel values are compressed to 512 numbers (73.5 : 1). Many different images map to similar vectors, so no exact inverse exists.

Tested on our own model. A direct inversion attack on our ArcFace model produced only noise, not a face.

Residual risk. Published attacks can generate look-alike faces from ArcFace embeddings so embeddings are still treated as sensitive biometric data under the DPDP Act, 2023.

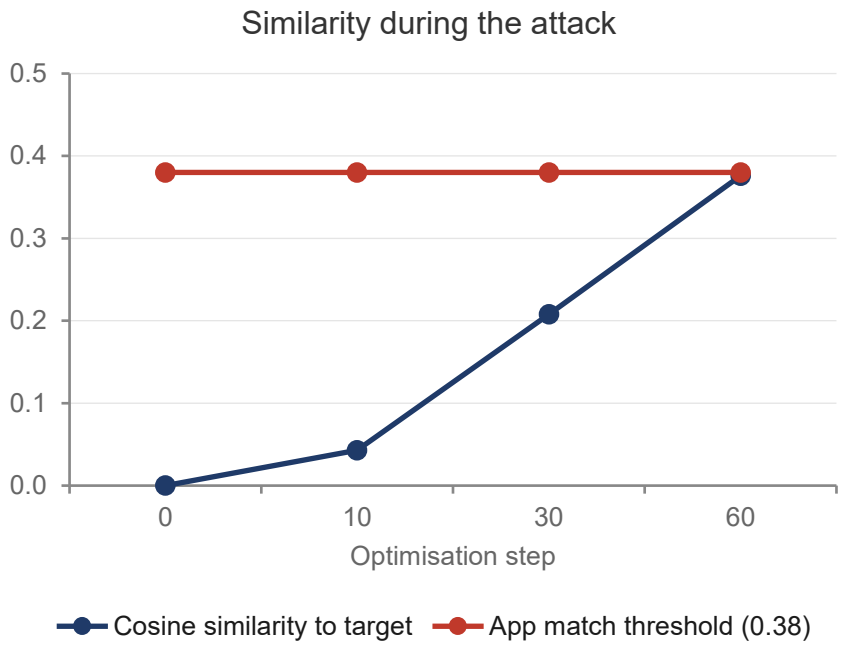
Safeguards

- No cloud upload; data leaves only when the teacher exports
- Stored in the app's private sandbox, protected by OS file encryption
- Planned: keystore-backed roster encryption and per-student deletion

Face Reconstruction Attack on Stored Embeddings

Attack setup

- Model: our arcface_mobilefacenet.onnx
- Target: a stored 512-D embedding of an enrolled student
- Method: black-box gradient ascent (Natural Evolution Strategies)
- Start: blank grey 112×112 image; 16 perturbations per step, 60 steps
- Goal: maximise cosine similarity between the image's embedding and the target



Findings

No face emerged: the image became high-frequency noise while similarity rose to 0.376

Why: 37,632 pixel values map to 512 numbers, so countless non-face images share a similar embedding

Mitigation: cancelable biometrics (BioHashing): store embeddings after a secret, per-class random projection, so a leaked template can be revoked and reissued.

Storage, Memory and Device Requirements

Storage	Size
iOS app build	**86.8 MB**
Bundled models (total)	**16.4 MB**
ArcFace MobileFaceNet	**13.6 MB**
RetinaFace / Haar / YuNet	**1.8 / 0.9 / 0.1 MB**
One embedding (binary)	**2 KB**
Per student: SQLite	**6 KB**
Per student: JSON	**≈ 32 KB**
60 students: SQLite	**≈ 0.4 MB**
60 students: JSON	**≈ 1.9 MB**

Memory (RAM)	Size
One 1080p frame	**6.2 MB**
Its 640×360 copy	**0.7 MB**
Frames per sweep (4 FPS)	**40 to 60**
Frames held in memory	**≈ 280 to 415 MB**
Models loaded	**≈ 16 MB**

Device	Minimum
iOS	**16 or later**
Android	**7.0 (API 24)+**
Processor	**ARM64 CPU**
GPU / NPU	**Not needed**
Rear camera	**1080p video**
Sensor	**Gyroscope**
RAM (recommended)	**4 GB**
Network	**Not needed**

Pipeline configuration

Sampling	Blur cut-off	YuNet score	Duplicate IoU	Cluster	Match (iOS / Android)
4 FPS	**15**	**0.40 / 0.35**	**0.35**	**0.35**	**0.40 / 0.38**

Sizes from the model files and code; JSON size estimated for 3 embeddings per student. RAM is estimated from the frames the code keeps in memory, not profiled on a phone. Tested on iPhone (iOS 26.6) and OnePlus CPH2661 (Android 16).

Benchmarking: Setup

System under test

- Android app: ML Kit detection, rear-row crop, desk grouping, ArcFace (ONNX Runtime), clustering and matching
- Default thresholds (cluster 0.35, match 0.38)

Device

- OnePlus CPH2661, Android 16, CPU only, offline

Classroom test

- One real classroom, 41 students enrolled
- All 41 students present during the sweep
- Ground truth: the actual roll for that lecture

Metrics

- Students marked present vs. actually present
- Attendance recall = correctly marked present ÷ actually present
- False-absence rate = present students marked absent

Benchmarking: Results

Base paper : static classroom photos on an NVIDIA T4 GPU

Model	Precision	Recall	F1	Time per image
Haar Cascade	0.92	0.32	0.47	0.66 s
MTCNN	0.99	0.92	0.96	6.14 s
YOLOFace	1.00	0.99	1.00	2.63 s
RetinaFace	0.99	1.00	1.00	10.75 s

Real classroom test (IOS, 41 students enrolled, all present)

41
students enrolled and present

27
marked present by the app

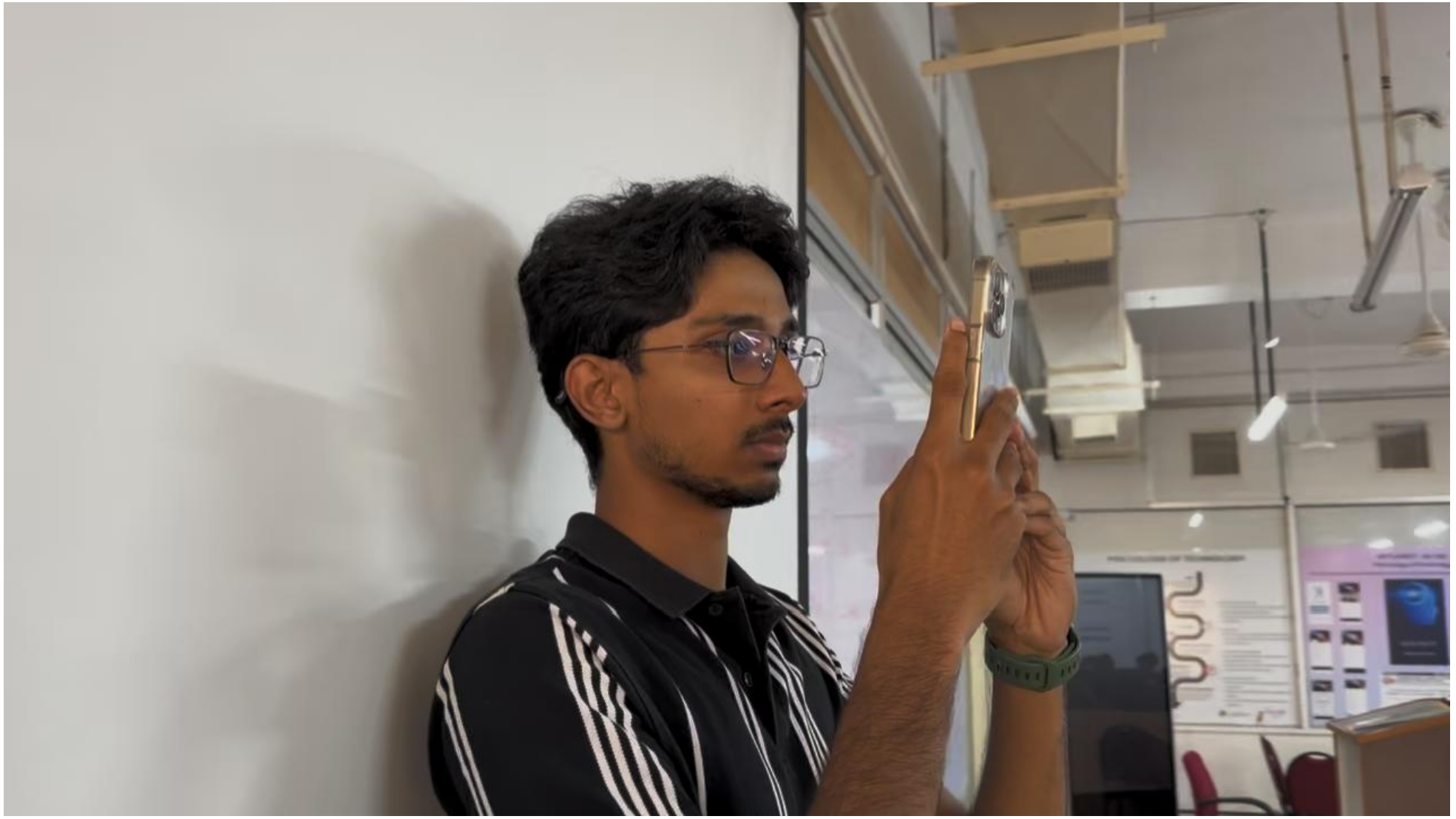
65.9%
attendance recall (27 / 41)

14
false absences (34.1%)

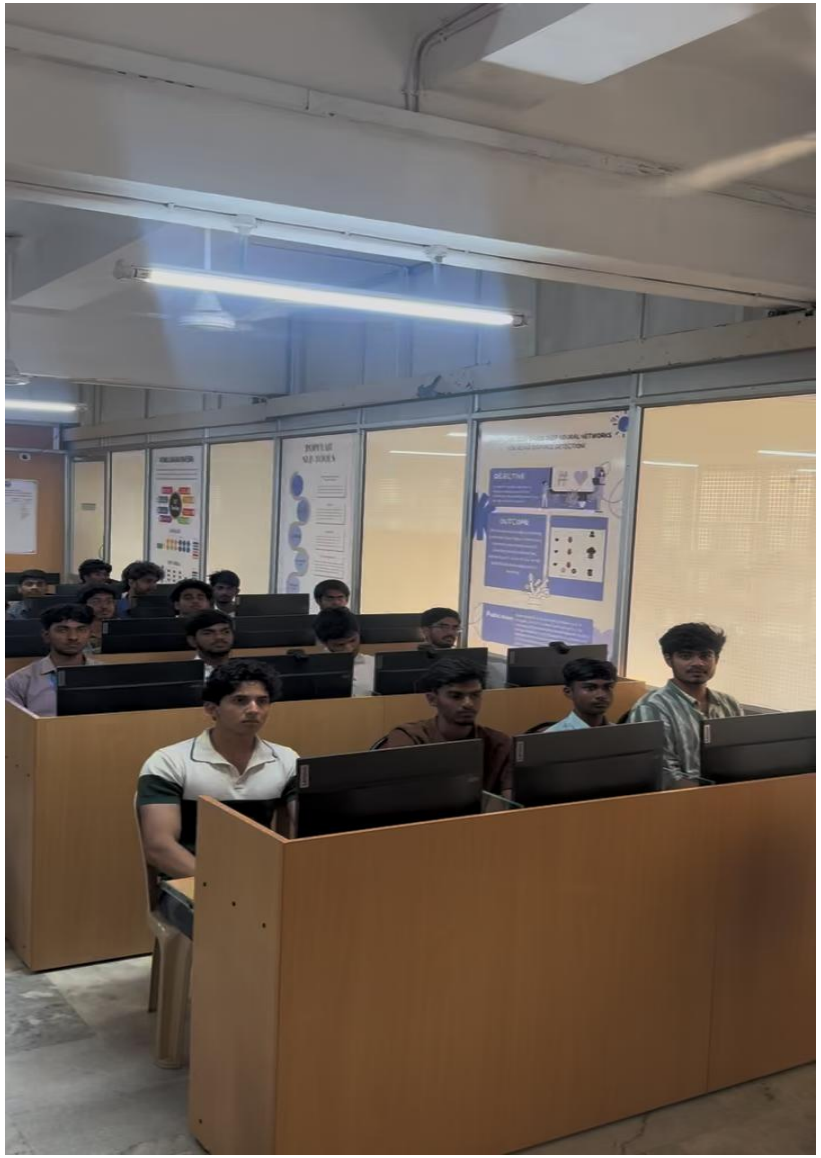
Every student marked present was actually present (no false presences, since all 41 attended). The 14 students the app missed are the main gap to close: their faces were not detected or matched with enough confidence in this sweep.

[1] G. F. Ananda, H. A. Nugroho and I. Ardiyanto, "Comparative analysis of multi-face detection methods in classroom environments," IEEE ICVEE, 2024.

Live Classroom Test



Live Classroom Test: Classroom Sweep



Live Classroom Test: Attendance Result

